AISES Software Evaluation Report

Executive Summary

Metric	Value
Total Requirements	12
Implemented	12
Partially Implemented	0
Not Implemented	0
Overall Compliance	100.0 %

Requirement Evaluation

AC-001 : Numeric Input and Multi-digit Numbers

Digits 0 through 9 append correctly to the input display when pressed.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Buttons for digits 0 through 9 are correctly mapped to the digit method with their respective string values.

• The digit method handles replacing initial zero/new input flags as well as appending subsequent digits properly.

Issues

None

Recommendations

None

AC-002 : Numeric Input and Multi-digit Numbers

Only one decimal point is allowed per operand.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The decimal method explicitly checks if '.' is already present in the current display value before appending a new decimal point.

Issues

None

Recommendations

None

AC-003 : Numeric Input and Multi-digit Numbers

Multi-digit numbers are supported and form correctly when consecutive digits are entered.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The digit method correctly handles string concatenation of consecutive digit inputs when self.new_input is False and the current display is not '0'.

Issues

None

Recommendations

None

AC-004 : Basic Arithmetic Operations

Addition operation computes the sum of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Addition operator is properly bound in the UI button data.

• The calculate method uses a dictionary mapping for operators including the '+' operation.

• Correctly handles float conversion for addition operands.

Issues

None

Recommendations

None

AC-005 : Basic Arithmetic Operations

Subtraction operation computes the difference of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The subtraction operation is correctly mapped to the '-' button and set_operator method.

• The operations dictionary correctly implements the lambda for subtraction using self.first - second.

• Result formatting correctly handles float representation and integer casting.

Issues

None

Recommendations

None

AC-006 : Basic Arithmetic Operations

Multiplication operation computes the product of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The multiplication operator '*' is mapped correctly in button data.

• The calculation method correctly uses the multiplication operator to compute the product of self.first and second operands.

Issues

None

Recommendations

None

AC-007 : Basic Arithmetic Operations

Division operation computes the quotient of operands correctly.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Division operation is correctly mapped to the '/' operator.

• The calculate method properly computes the quotient using self.first / second.

• Zero division is explicitly handled by catching ZeroDivisionError and displaying an appropriate message.

Issues

None

Recommendations

None

AC-008 : Clear State

Clear (C) button resets all calculator state and display values.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The 'C' button is correctly mapped to the clear method in button_data.

• The clear method resets the display value to '0', first operand to 0.0, operator to None, and new_input to True, fully resetting calculator state.

Issues

None

Recommendations

None

AC-009 : Delete Last Character

Delete (DEL) button removes the last entered character from the input display.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The DEL button is correctly mapped to the delete method.

• The delete method properly slices off the last character using current[:-1] when length is greater than 1.

• The delete method falls back to '0' and resets new_input when only one character remains.

Issues

None

Recommendations

None

AC-010 : Division by Zero Validation

Division by zero is detected before calculation and an error message is displayed instead of performing the calculation.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Division by zero is successfully caught using a try-except block handling ZeroDivisionError.

• An error message 'Cannot divide by zero' is displayed on the calculator interface instead of crashing or performing the calculation.

Issues

None

Recommendations

None

AC-011 : Evaluate Expression

Pressing '=' computes the result of the current expression.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• The '=' button is successfully bound to the calculate method in the button mapping data.

• The calculate method handles basic arithmetic operations (+, -, *, /) correctly.

• Proper handling of division by zero and formatting of integer results.

Issues

None

Recommendations

None

AC-012 : Display Formatted Numeric Results

Results are formatted correctly and presented clearly on the display field.

Status : Implemented

Coverage : 100 %

Confidence : 100

Matched Files

• Calculator.py

Strengths

• Results are explicitly formatted to remove unnecessary floating-point decimals when the value is an integer.

• Errors such as division by zero are caught and presented clearly as text messages on the display field.

Issues

None

Recommendations

None